

# **Compilador MiniJava**

Etapa 1

**Implementación del Análisis Léxico**

Compiladores e Intérpretes  
Agosto de 2025

**Ludmila Prolygin**

|                                                  |          |
|--------------------------------------------------|----------|
| <b>Tabla de contenidos</b>                       |          |
| <b>Compilación y ejecución del código fuente</b> | <b>2</b> |
| <b>Reconocimiento de tokens</b>                  | <b>2</b> |
| Palabras reservadas                              | 2        |
| Operadores                                       | 3        |
| Puntuación                                       | 3        |
| <b>Detección de errores léxicos</b>              | <b>3</b> |
| <b>Decisiones de diseño</b>                      | <b>3</b> |
| Clases                                           | 3        |
| Paquetes                                         | 4        |
| Consideraciones adicionales                      | 4        |
| <b>Logros</b>                                    | <b>4</b> |
| <b>Repositorio</b>                               | <b>4</b> |

## Compilación y ejecución del código fuente

Para realizar la compilación del código fuente entregado, se utiliza el siguiente comando por consola:

```
javac -sourcepath src -d out src/main/Main.java
```

En el caso de la ejecución, para invocar el programa, se utiliza

```
java -cp out main.Main <archivo de test>
```

Se incluye también un archivo .jar que puede ser ejecutado con

```
java -jar Compilador.java <fuente.java>
```

## Reconocimiento de tokens

***END\_OF\_FILE***

EOF

***idClase***

$([a..z])([a..z, A..Z, 0..9, _])^*$

***idMetVar***

$([A..Z])([a..z, A..Z, 0..9, _])^*$

***intLiteral***

$([0..9])^1 \mid ([0..9])^2 \mid ([0..9])^3 \mid$   
 $([0..9])^4 \mid ([0..9])^5 \mid ([0..9])^6 \mid$   
 $([0..9])^7 \mid ([0..9])^8 \mid ([0..9])^9$

***charLiteral***

$'([x \in \gamma])' \mid '(\backslash[x \in \gamma])' \mid '(\backslash[x \in \gamma]^4)'$

Siendo  $\gamma$  el conjunto de todos los caracteres ASCII disponibles.

***stringLiteral***

$"([x \in \gamma]^*)"$

Siendo  $\gamma$  el conjunto de todos los caracteres ASCII disponibles.

### Palabras reservadas

|         |        |       |          |
|---------|--------|-------|----------|
| class   | final  | this  | char     |
| extends | if     | new   | int      |
| public  | else   | null  | abstract |
| static  | while  | true  | var      |
| void    | return | false | boolean  |

## Operadores

|    |    |   |    |
|----|----|---|----|
| >  | -  |   | == |
| <  | ++ | % | !  |
| >= | /  | * | != |
| <= | -- | + | && |

## Puntuación

|   |   |   |   |
|---|---|---|---|
| ( | { | ; | : |
| ) | } | , | . |

## Detección de errores léxicos

El analizador léxico tiene la capacidad de detectar los errores mencionados a continuación:

- `invalidSymbol`. Símbolos no válidos conforme a la especificación técnica del lenguaje.
- `integerTooLong`. Números enteros de mayor largo del permitido, es decir, números de más de 9 dígitos de longitud.
- `invalidCharacter`. Caracteres mal compuestos; más de un carácter entre apóstrofes, sin caracteres entre apóstrofes, caracteres sin cerrar.
- `invalidUnicodeCharacter`. Construcción errónea de caracteres unicode; caracteres que comienzan con `\u` pero no finalizan una codificación correcta.
- `invalidMultilineComment`. Comentarios multilínea que no están cerrados con el correspondiente `*/`.
- `invalidCarriageReturn`. Usos de retorno de carro en medio de *stringLiterals* o *charLiterals*.
- `invalidEndOfFile`. Finalización del archivo en medio de *stringLiterals* o *charLiterals*.
- `invalidAmpersandUsage`. Uso individual del carácter `&`; se detecta la individualidad y sugiere el cambio por `&&`.
- `invalidPipeUsage`. Uso individual del carácter `|`; se detecta la individualidad y sugiere el cambio por `||`.

## Decisiones de diseño

### Clases

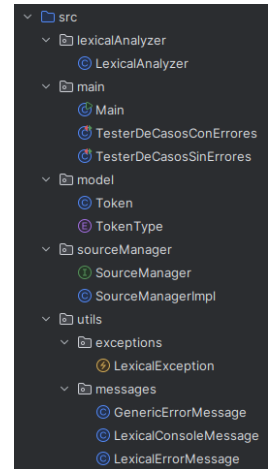
- `LexicalAnalyzer`. Esta clase se encarga de manejar el pasaje de estados de los autómatas que describen el comportamiento del lenguaje.
- `TokenType`. Enumerado que identifica los diferentes tipos de tokens disponibles en el lenguaje. Cada uno de ellos se vincula con una cadena de caracteres descriptiva del tipo.
- `LexicalException`. Modela las excepciones léxicas.

## Paquetes

- `main`. Incluye todas las clases ejecutables.
- `model`. Contiene a las clases `Token` y `TokenType` que describen los terminales del lenguaje.
- `sourceManager`. La interfaz y clase que la implementa, provistas por la cátedra, están incluidas en este paquete.

## Consideraciones adicionales

- La clase `LexicalAnalyzer` incluye un método `init(SourceManager sm)` que asocia el objeto de tipo `SourceManager` parametrizado y, además, inicializa el carácter actual (`currentChar`). Se decidió utilizar esta estrategia para realizarlo puesto que es necesario realizar la apertura del archivo conteniendo los casos de prueba para inicializar el carácter actual y poder continuar con el análisis léxico.



## Logros

Se intentaron alcanzar los siguientes logros:

- entrega anticipada léxica,
- imbatibilidad léxica,
- unicodes,
- reporte de error elegante,
- columnas,
- multi detección de errores léxicos.

El logro de manejador de archivos eficiente no fue implementado.

## Repositorio

<https://github.com/ludmilapropygin/compiladorMiniJava/tree/Etapa1>